Research Memo: Off-Chain Bot vs. Solidity Bot

Project anchored to **QQQ_Dynamic_Leverage_Band_Model_v1.xlsx**

The current project already contains a precise strategy rule in the workbook. In the **Model** sheet, the dynamic strategy is not “buy TQQQ” or “buy SQQQ.” It is a directly leveraged **QQQ** position with borrowed cash, a target leverage of **3.0**, and a rebalance band of **[3.0, 3.3]**. The operative state is carried by columns **T:AG**, especially **Shares_prev**, **Cash_pre**, **Equity_pre**, **Leverage_pre**, **Rebalance_Action**, **Target_Pos**, **Trade_Shares**, **Shares_new**, **Cash_new**, and **Leverage_new**. Therefore, the strategy is already specified; the research question is which implementation architecture best realizes it and best supports reasoning about correctness.

An **off-chain bot** is an ordinary program, typically in Python or JavaScript, that runs outside the blockchain and interacts with a broker or exchange through APIs. This style matches the workbook very naturally: on each step the bot reads the latest price and current portfolio state, computes leverage, checks whether the portfolio is below 3.0 or above 3.3, and submits a buy or sell order if needed. Its main advantage is **practicality**: it is easy to schedule, backtest, integrate with APIs, and eventually deploy. Its main disadvantage is that correctness is harder to state cleanly, because actual behavior depends on network timing, API semantics, order fills, and operational failures in addition to the rebalance formula itself.

A **Solidity bot** or smart-contract prototype puts the state transition on chain. This is attractive for the present project because Professor Xi’s question is not only how to automate trading, but whether one can show in a mathematical sense that an implementation matches its intended specification. Solidity is a better fit for that question because the state variables and transition rules are explicit, and the implementation can include **assert**-style correctness checks. The main disadvantage is that a contract is not a full live trading system by itself: it does not fetch market prices on its own and it does not wake itself up on a schedule. A real deployment would still need an external price feed and an external caller or automation layer.

For this project, the cleanest path is to treat the workbook as the **source specification**, write a short formal specification of the state transition, and then build a **minimal Solidity prototype** that mirrors the workbook rule as closely as possible. This keeps continuity with the existing QQQ model while moving the work toward the professor’s real research goal: **specification - implementation - correctness**. Off-chain automation can later be added as a deployment layer, but Solidity is the better next step for a correctness-oriented prototype.